



Computer Vision for Autonomous Vehicles

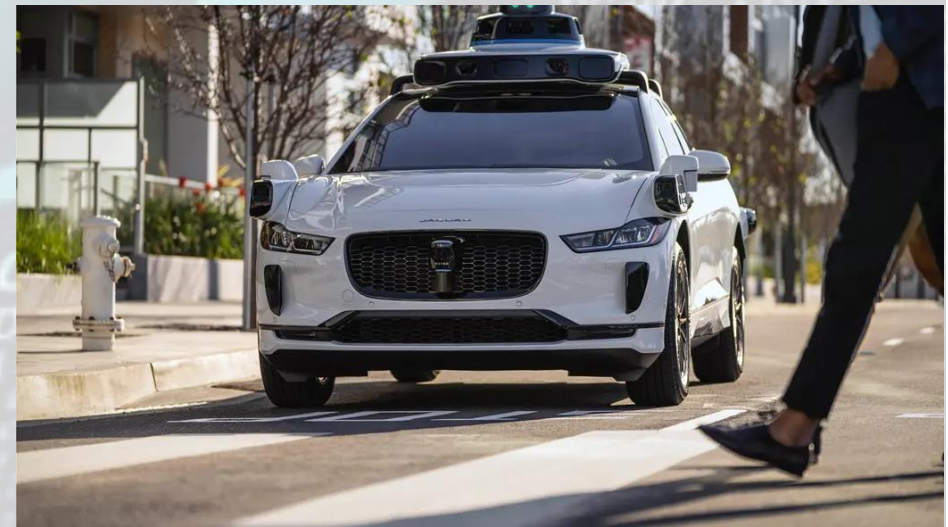
Lecture 3: 3D Geometry

Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2025

Sept 2nd, 2025



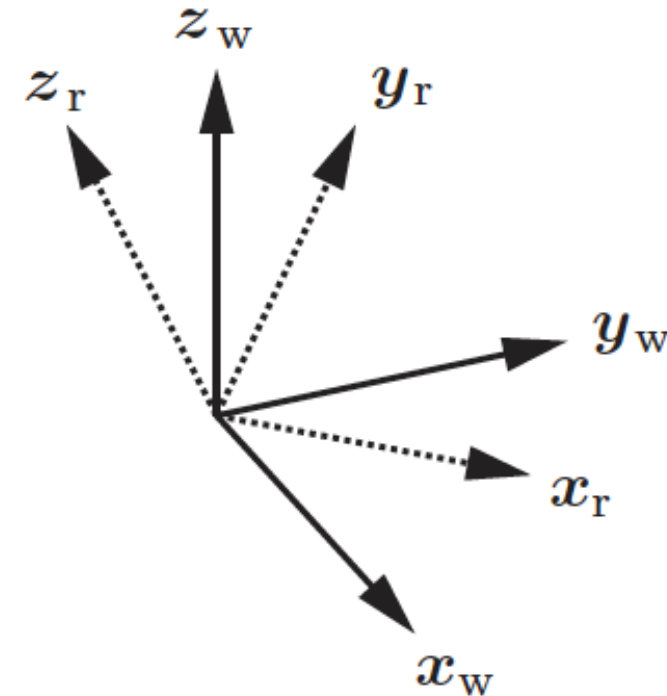
Announcement

- Lab 2 (ROS) will be posted by tomorrow. Deadline, Wed Sept 10.
- Deadline for team assignment, tomorrow
- Plan for releasing robots: This week

3D Rotation Matrix

- Each column with the of the axes of r expressed in the coordinate frame w

$$R_r^w = \begin{bmatrix} x_r^w & y_r^w & z_r^w \end{bmatrix}$$



Rotation Matrix Conditions

■ Orthogonality

orthogonality: the axes $\mathbf{x}_r^w$, $\mathbf{y}_r^w$, $\mathbf{z}_r^w$ have unit length and are orthogonal to each other (independently on the reference frame they are expressed in), therefore:

$$\begin{aligned} \|\mathbf{x}_r^w\|_2^2 &= 1 & \|\mathbf{y}_r^w\|_2^2 &= 1 & \|\mathbf{z}_r^w\|_2^2 &= 1 & \text{(unit length)} & (2.7) \\ (\mathbf{x}_r^w)^\top \mathbf{y}_r^w &= 0 & (\mathbf{x}_r^w)^\top \mathbf{z}_r^w &= 0 & (\mathbf{y}_r^w)^\top \mathbf{z}_r^w &= 0 & \text{(orthogonal vectors)} & (2.8) \end{aligned}$$

■ Right-handedness

right-handedness: the 3D axes $\mathbf{x}_r^w$, $\mathbf{y}_r^w$, $\mathbf{z}_r^w$ have to satisfy the right-hand rule, which implies that $\mathbf{x}_r^w \times \mathbf{y}_r^w = \mathbf{z}_r^w$ where $\times$ denotes the cross product between vectors. Since these are unit-length vectors, the following relations hold:

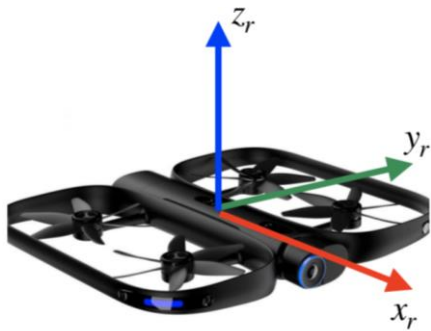
$$\mathbf{x}_r^w \times \mathbf{y}_r^w = \mathbf{z}_r^w \iff (\mathbf{z}_r^w)^\top (\mathbf{x}_r^w \times \mathbf{y}_r^w) = +1 \iff \det(\mathbf{R}_r^w) = +1 \quad (2.11)$$

Example

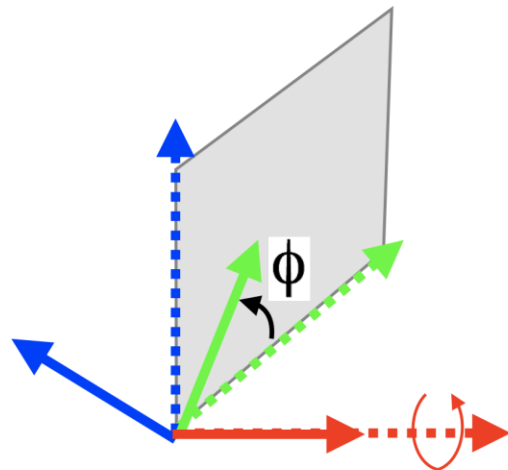
- Consider $x_r = [0 \ 0 \ 1]$, $y_r = [-1 \ 0 \ 0]$, and $z_r = [0 \ -1 \ 0]$, are axes of robot body in the world coordinate. What is $R_r^w = ?$
- Solution:
- $R_r^w = [x_r \ y_r \ z_r]$, but we need to first check if they meet other requirement:
- Normalized?
- Orthogonal?
- Right-handedness?

Elementary rotations and Euler angles representation

- **Theorem 1 (Euler's rotation theorem, 1775).** *Any rotation can be written as the product of no more than three elementary rotations (no two consecutive rotations along the same axis), where the elementary rotations are defined as follows:*

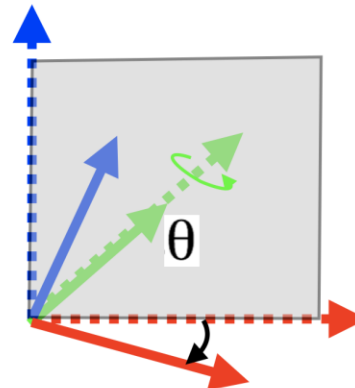


Rotation around
the **x** axis



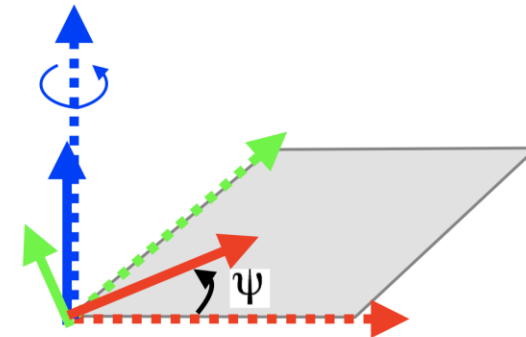
$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}$$

Rotation around
the **y** axis



$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

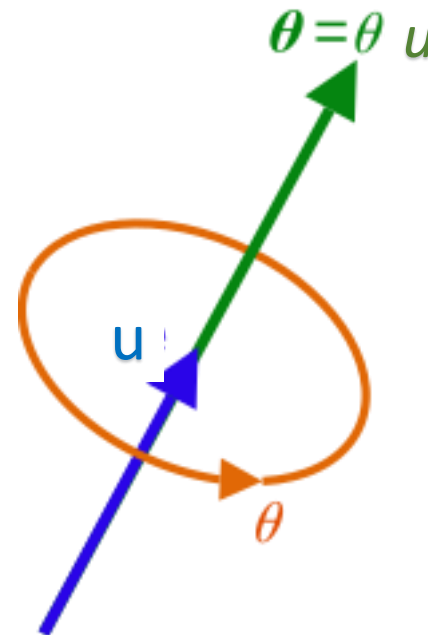
Rotation around
the **z** axis



$$R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Axis-angle representation

- *Theorem 2 (Euler's rotation theorem, 1776). Every rotation can be described as a rotation of an angle θ around an axis u (with $\|u\| = 1$), not necessarily aligned with the Cartesian axes.*
- indicating the direction (geometry) of an axis of rotation, and an angle of rotation ϑ describing the magnitude and sense (e.g., clockwise) of the rotation about the axis.
- **u : rotation vector**, also called the **Euler vector**
- Equivalent to a pure rotation about a single fixed axis
- **Properties:**
- Encodes a 3D rotation using the pair (u, θ)
- Only stores Four parameters (three if we use $\|u\| = 1$)



Conversion [optional]

- **From axis-angle to rotation matrix (Rodrigues' rotation formula):**
- Given a rotation angle θ and a rotation axis u that describe the attitude of the frame r with respect to the frame w (with u expressed in the frame w), the rotation matrix R_r^w can be computed as:

$$R_r^w = \cos(\theta)\mathbf{I}_3 + \sin(\theta)[u]_{\times} + (1 - \cos(\theta))uu^T$$

$$[u]_{\times} \doteq \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}$$

- $[.]_{\times}$ is skew symmetric matrix (cross product matrix).
- **Note:** using cross product matrix we can write the cross product between 2 vectors v_1 and v_2 as:

$$v_1 \times v_2 = [v_1]_{\times} v_2$$

Conversion [Optional]

- *From rotation matrix to axis-angle:* given a rotation matrix R_r^w that describes the attitude of the frame r with respect to the frame w , the corresponding axis-angle representation (u, θ) can be computed as:

- Angle

$$\text{tr}(\mathbf{R}_r^w) = 1 + 2 \cos(\theta) \quad \longrightarrow \quad \theta = \arccos\left(\frac{\text{tr}(\mathbf{R}_r^w) - 1}{2}\right)$$

- Axis u

- Since any vector on u axis does not change under any rotation around this axis: $\mathbf{R}_r^w u = u$

- Which means u is the eigen vector of Rotation matrix for the eigen value 1
- **To compute u :** compute the eigen vector of rotation matrix corresponding to eigen value 1

Quaternion representation

- Extends complex numbers for expressing spatial rotation in 3D
- First introduced by William Hamilton (1843)
- A quaternion is denoted as a column vector

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix}$$

or as an hypercomplex number, i.e., $\mathbf{q} = iq_1 + jq_2 + kq_3 + q_4$, with i, j, k satisfying:

$$\begin{aligned} i^2 &= j^2 = k^2 = ijk = -1 \\ ij &= -ji = k & jk &= -kj = i & ki &= -ik = j \end{aligned}$$

Conversion between Quaternion and other rotation representation

- **Unit quaternions:** quaternions with unit norm
- From Axis-angle to quaternion and Vice versa:

$$\mathbf{q} = \begin{bmatrix} \sin(\theta/2)\mathbf{u} \\ \cos(\theta/2) \end{bmatrix}$$

- From Unit Quaternion to Rotation matrix:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} q_1^2 - q_2^2 - q_3^2 + q_4^2 & 2(q_1q_2 - q_3q_4) & 2(q_1q_3 + q_2q_4) \\ 2(q_1q_2 + q_3q_4) & -q_1^2 + q_2^2 - q_3^2 + q_4^2 & 2(q_2q_3 - q_1q_4) \\ 2(q_1q_3 - q_2q_4) & 2(q_2q_3 + q_1q_4) & -q_1^2 - q_2^2 + q_3^2 + q_4^2 \end{bmatrix}$$

$$\mathbf{R}(\mathbf{q}) = \mathbf{R}(-\mathbf{q})$$

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix}$$

Quaternion Product

Quaternion composition. Given two quaternions $\mathbf{q}_a = [q_{a,1} \ q_{a,2} \ q_{a,3} \ q_{a,4}]^T$ and $\mathbf{q}_b = [q_{b,1} \ q_{b,2} \ q_{b,3} \ q_{b,4}]^T$, we define the *quaternion product* $\mathbf{q}_c = \mathbf{q}_a \otimes \mathbf{q}_b$, which can be computed as:

$$\mathbf{q}_c = \begin{bmatrix} q_{a,4} & -q_{a,3} & q_{a,2} & q_{a,1} \\ q_{a,3} & q_{a,4} & -q_{a,1} & q_{a,2} \\ -q_{a,2} & q_{a,1} & q_{a,4} & q_{a,3} \\ -q_{a,1} & -q_{a,2} & -q_{a,3} & q_{a,4} \end{bmatrix} \begin{bmatrix} q_{b,1} \\ q_{b,2} \\ q_{b,3} \\ q_{b,4} \end{bmatrix} = \Upsilon(\mathbf{q}_a) \mathbf{q}_b$$

or, equivalently, as:

$$\mathbf{q}_c = \begin{bmatrix} q_{b,4} & q_{b,3} & -q_{b,2} & q_{b,1} \\ -q_{b,3} & q_{b,4} & q_{b,1} & q_{b,2} \\ q_{b,2} & -q_{b,1} & q_{b,4} & q_{b,3} \\ -q_{b,1} & -q_{b,2} & -q_{b,3} & q_{b,4} \end{bmatrix} \begin{bmatrix} q_{a,1} \\ q_{a,2} \\ q_{a,3} \\ q_{a,4} \end{bmatrix} = \bar{\Upsilon}(\mathbf{q}_b) \mathbf{q}_a$$

Quaternion Composition and inverse

Also, let us assume we are given the attitude of a frame r with respect to a frame w as a quaternion, namely q_r^w , and the attitude of a frame c with respect to a frame r , namely q_c^r . How can we compute the attitude of the frame c with respect to the frame w , i.e., q_c^w ?

It turns out that we can compose rotations using the quaternion product introduced above:

$$q_c^w = q_r^w \otimes q_c^r \quad (2.37)$$

Inverse of a quaternion. Let us assume we are given the attitude of a frame r with respect to a frame w in quaternion notation, namely q_r^w . This section addresses the following question: how can we compute the attitude of w with respect to frame r , i.e., q_w^r ?

$$q = [v^T \ s]^T \quad q_w^r = (q_r^w)^{-1} = \begin{bmatrix} v_r^w \\ s_r^w \end{bmatrix}^{-1} = \begin{bmatrix} -v_r^w \\ s_r^w \end{bmatrix}$$

$$(q_a \otimes q_b \otimes \dots \otimes q_N)^{-1} = q_N^{-1} \otimes \dots \otimes q_b^{-1} \otimes q_a^{-1}$$

Expressing points in a rotated frame in Quaternion

- Assume $\mathbf{p}^r$ is a point expressed in frame r
- Frame r is rotated respect to the world frame w ,
- rotation is defined in the quaternion $\mathbf{q}_r^w$
- Question: what is the expression of point p in w $\mathbf{p}^w = ?$
- **Solution: two different ways of expressing point p in frame w :**
- We consider homogenous coordinate (later), adding extra dimension (0 or 1):
- **Note:** depending on the format of rotation matrix representation (matrix or quaternion), we can express the point

$$\mathbf{p}^w = (\mathbf{q}_r^w) \otimes \begin{bmatrix} \mathbf{p}^r \\ 1 \end{bmatrix} \otimes (\mathbf{q}_r^w)^{-1} = \begin{bmatrix} \mathbf{R}_r^w \mathbf{p}^r \\ 1 \end{bmatrix}$$
$$\mathbf{p}^w = (\mathbf{q}_r^w) \otimes \begin{bmatrix} \mathbf{p}^r \\ 0 \end{bmatrix} \otimes (\mathbf{q}_r^w)^{-1} = \begin{bmatrix} \mathbf{R}_r^w \mathbf{p}^r \\ 0 \end{bmatrix}$$

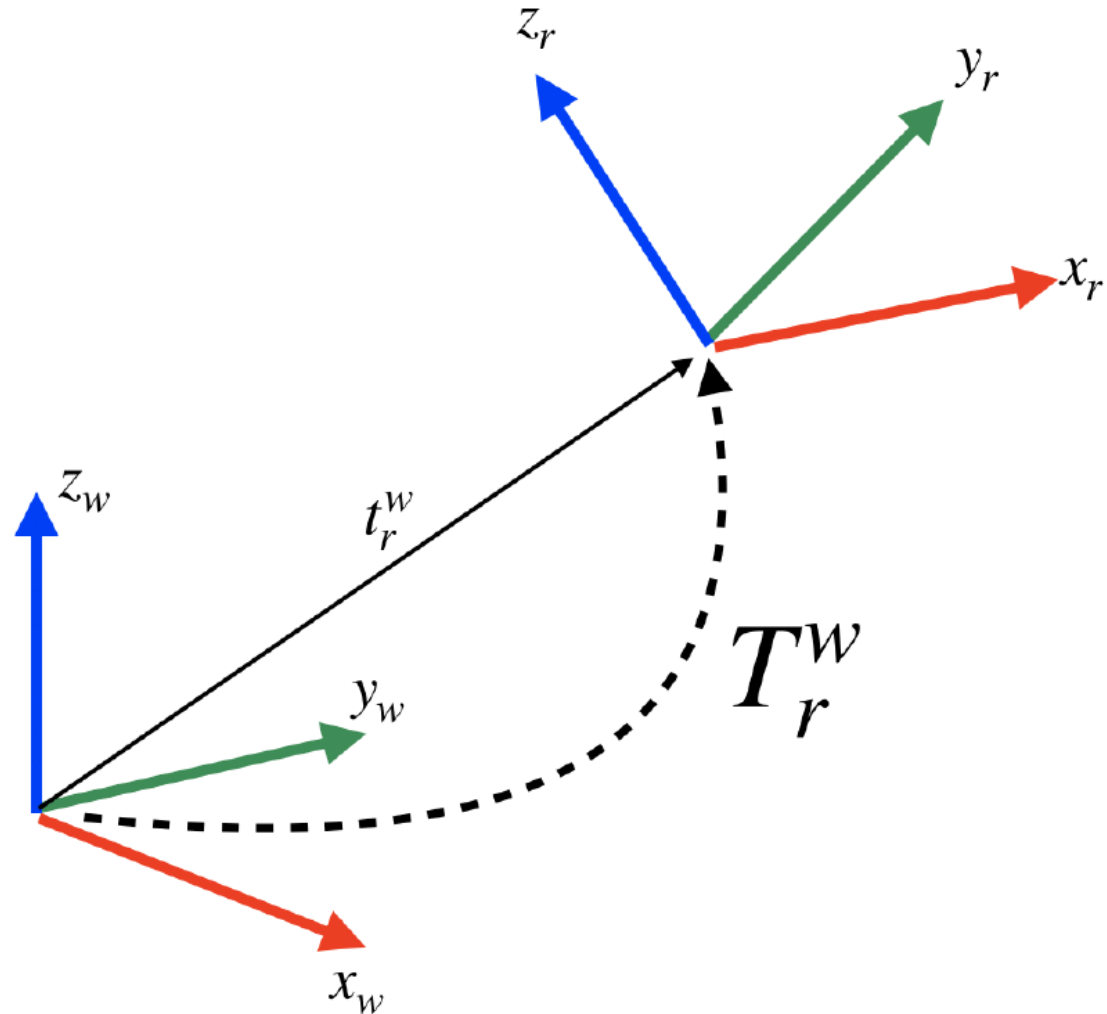
Poses and rigid-body transformations

Combination of rotation and translation

$$(\mathbf{R}_r^w, \mathbf{t}_r^w)$$

$$\mathbf{p}^w = \mathbf{R}_r^w \mathbf{p}^r + \mathbf{t}_r^w$$

$$\mathbf{T}_r^w = \begin{bmatrix} \mathbf{R}_r^w & \mathbf{t}_r^w \\ \mathbf{0}_d^\top & 1 \end{bmatrix}$$



Homogeneous Coordinate Representation

$$\tilde{\mathbf{p}}^r = \begin{bmatrix} \mathbf{p}^r \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{p}}^w = \begin{bmatrix} \mathbf{R}_r^w & \mathbf{t}_r^w \\ \mathbf{0}_d^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{p}^r \\ 1 \end{bmatrix} = \mathbf{T}_r^w \tilde{\mathbf{p}}^r$$

Pose composition

$$\mathbf{T}_c^w = \mathbf{T}_r^w \mathbf{T}_c^r$$

Inverse a pose

$$\mathbf{T}_w^r = (\mathbf{T}_r^w)^{-1} = \begin{bmatrix} (\mathbf{R}_r^w)^\top & -(\mathbf{R}_r^w)^\top \mathbf{t}_r^w \\ \mathbf{0}_3^\top & 1 \end{bmatrix}$$

Let's Practice!

1. Find the quaternion representation r_1 and matrix representation R_1 of the rotation by $\pi/2$ around x-axis.
2. Find the quaternion representations r_2 and matrix representation R_2 of the rotation by $\pi/2$ around y-axis.
3. Construct the quaternion representations r_1^2 of the rotation r_1^2 (hint: $r_1^2 = (r_2^w)^{-1} \otimes r_1^w$)
4. Find the rotation matrix $R_1^2 = (R_2)^{-1} R_1$ by matrix multiplication.
5. Construct the rotation matrix from r_1^2 and compare it to R_1^2 .
6. Construct the quaternion from the rotation matrix R_1^2 and compare it to r_1^2 . (Hint: $R \rightarrow \text{axis \& angle} \rightarrow \text{quaternion}$).

Introduction to ROS - please read the intro page in canvas carefully before starting your assignment!

- Robot Operating System
- Communication between the ROS modules called nodes.
- Those nodes can be executed on a single machine or across several machines, obtaining a distributed system.
- The advantage of ROS structure
 - Each node can control one aspect of a system.
 - For example you might have several nodes each be responsible of parsing row data from sensors and one node to process them.

Introduction to ROS

